

The Transformer Architecture

A comprehensive visual guide to the model that reshaped modern AI — from self-attention and positional encoding to multi-head attention and the decoder stack.



Uditya Narayan Tiwari
Technical Author

Vaswani et al., 2017

Attention Is All You Need

Parallelizable

OVERVIEW

ENCODER

SELF-ATTENTION

MULTI-HEAD

POSITIONAL ENC.

RESIDUALS

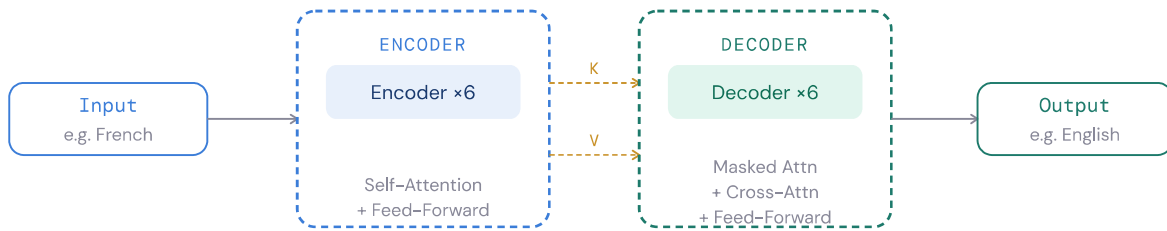
DECODE

01 — HIGH-LEVEL VIEW

What Is the Transformer?

The Transformer is a neural network architecture introduced in the 2017 paper *Attention Is All You Need*. Unlike recurrent models (RNNs/LSTMs) that process tokens sequentially, the Transformer processes all positions simultaneously — making it highly parallelizable and dramatically faster to train.

At the macro level, the model is an encoder–decoder stack. Given an input sentence, the encoder builds rich contextual representations; the decoder autoregressively generates the output sequence one token at a time.



`d_model = 512`

6 encoder layers

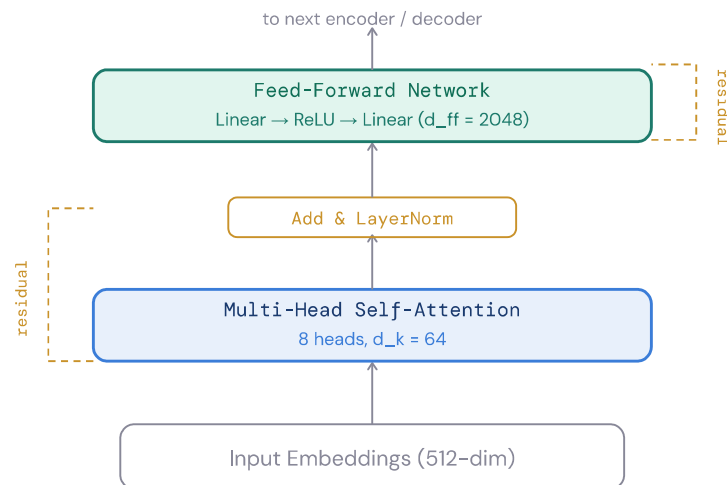
6 decoder layers

8 attention heads

02 — ENCODER BLOCK

Inside One Encoder Layer

Each of the six encoders is structurally identical (though weights are not shared). Every encoder consists of exactly two sub-layers.



The feed-forward sub-layer uses the same network independently at each position — all positions in a sequence are processed in parallel, which is the key source of the Transformer's training speed advantage over RNNs.

Scaled Dot-Product Attention

Self-attention lets every token look at every other token in the sequence to build a richer representation. For each token, three vectors are derived: a Query (Q), a Key (K), and a Value (V), each of dimension `d_k = 64`.

CORE EQUATION

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}} \right) \cdot V$$

where $Q \in \mathbb{R}^{n \times d_k}$, $K \in \mathbb{R}^{n \times d_k}$, $V \in \mathbb{R}^{n \times d_v}$, and $d_k = 64$

Six Steps, Computed in Parallel

STEP 01

Project to Q, K, V

Multiply each embedding x_i (512-dim) by learned weight matrices W^Q, W^K, W^V to get 64-dim query, key, and value vectors.

STEP 02

Compute raw scores

$\text{Score}(q_i, k_j) = q_i \cdot k_j$ — the dot product of the query of token i with the key of token j , for every j in the sequence.

STEP 03-04

Scale & Softmax

Divide scores by $\sqrt{d_k} = 8$ for stable gradients. Pass through softmax so all scores are positive and sum to 1, producing attention weights α_{ij} .

STEP 05-06

Weighted sum of Values

Multiply each value vector v_j by its weight α_{ij} , then sum: $z_i = \sum \alpha_{ij} \cdot v_j$. This is the attention output for position i .

MATRIX FORM

$$Q = XW^Q \quad \cdot \quad K = XW^K \quad \cdot \quad V = XW^V$$

$$X \in \mathbb{R}^{n \times 512} \rightarrow Q, K \in \mathbb{R}^{n \times 64} \rightarrow V \in \mathbb{R}^{n \times 64}$$

When encoding the word "it" in "The animal didn't cross the street because it was too tired", self-attention assigns high weight to "animal" — the model learns to resolve coreference entirely through the attention mechanism, without any hand-crafted rules.

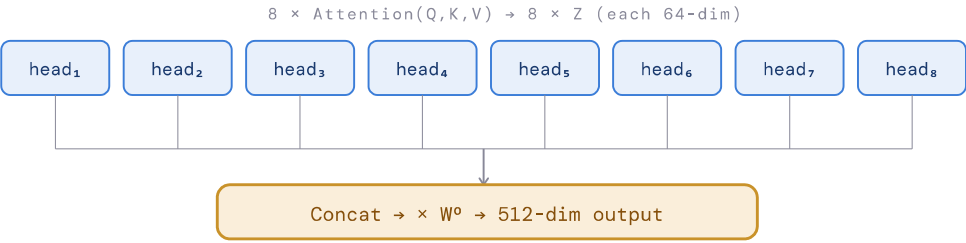
Running Attention in Parallel Subspaces

Instead of a single attention pass, the Transformer uses 8 attention heads. Each head has its own W^Q, W^K, W^V matrices, projecting into a different 64-dimensional subspace. The eight outputs $Z_0...Z_7$ are concatenated and linearly projected back to 512 dimensions.

MULTI-HEAD EQUATION

$$\text{MultiHead}(Q,K,V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) W^O$$

where $\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$



PARAMETER	SINGLE-HEAD	MULTI-HEAD (H=8)
Attention dim per head	512	64 (512 / 8)
W^Q, W^K, W^V matrices	1 each	8 each (independent)
Representation subspaces	1	8 different subspaces
Output projection W^O	None needed	$\mathbb{R}^{512 \times 512}$
Total computation	Equivalent	Equivalent (by design)

Injecting Order Into the Sequence

Since self-attention has no inherent notion of order, the Transformer adds a positional encoding vector to each word embedding before the first encoder. These vectors follow a sinusoidal pattern, giving the model a continuous, generalizable sense of position.

SINUSOIDAL ENCODING

$$PE_{(pos, 2i)} = \sin(pos / 10000^{2i/d_{model}})$$

$$PE_{(pos, 2i+1)} = \cos(pos / 10000^{2i/d_{model}})$$

pos = position in sequence, i = dimension index, d_model = 512

The even dimensions use sine and odd dimensions use cosine. The resulting 512-dim encoding is added to the embedding: $\hat{x}_{pos} = x_{pos} + PE_{pos}$. This approach generalizes to sequence lengths not seen during training.



06 — RESIDUAL CONNECTIONS & LAYER NORM

Add & Norm Wrappers

Every sub-layer is wrapped with a residual connection followed by layer normalization. This stabilizes gradients and allows the network to be trained deeply.

SUB-LAYER WRAPPER

$$\text{Output} = \text{LayerNorm}(x + \text{Sublayer}(x))$$

where Sublayer(x) is either the self-attention or the FFN function

LAYER NORMALIZATION

$$\text{LayerNorm}(x) = \gamma \cdot (x - \mu) / (\sigma + \epsilon) + \beta$$

μ = mean over features, σ = std deviation, γ and β are learned scale/shift parameters, $\epsilon \approx 1e-5$

The residual path means gradients flow directly through addition — bypassing the sub-layer entirely — which addresses the vanishing-gradient problem that plagued deep networks. Combined with layer norm, this enables stable training of 6-layer (or deeper) Transformer stacks.

07 — DECODER STACK

Autoregressive Generation

The decoder has three sub-layers per block. It extends the encoder with a cross-attention layer that lets each decoder position attend to the full encoder output.

SUB-LAYER 1

Masked Self-Attention

Like encoder self-attention, but tokens can only attend to previous positions. Future positions are masked (set to $-\infty$ before softmax) to preserve the autoregressive property.

SUB-LAYER 2

Encoder-Decoder Attention

Queries come from the decoder; Keys and Values come from the encoder's final output. This is how the decoder "reads" the source sentence at each generation step.

SUB-LAYER 3

Position-wise Feed-Forward Network

Identical to the encoder FFN — two linear transformations with ReLU activation, $d_{ff} = 2048$. Applied independently to each position. Followed by Add & Norm.

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{(QK^T + M)}{\sqrt{d_k}}\right) \cdot V$$

MASKED ATTENTION

Mask $M_{ij} = 0$ if $j \leq i$, else $-\infty$ (ensures token i cannot see token $j > i$)

08 — OUTPUT LAYER

From Vector to Word

After the decoder stack, a linear projection maps the 512-dim decoder output into a logit vector of size $|V|$ (vocabulary size). A softmax converts logits to probabilities, and the highest-probability token is emitted.

$$\text{logits} = W_{\text{out}} \cdot h_{\text{decoder}} + b$$

OUTPUT PROJECTION

$W_{\text{out}} \in \mathbb{R}^{|V| \times 512}$, shared (tied) with the input embedding matrix in many implementations

$$P(\text{token}_i) = \exp(\text{logit}_i) / \sum_j \exp(\text{logit}_j)$$

TOKEN PROBABILITY

Greedy decoding picks argmax; beam search maintains top-k hypotheses

$$\mathcal{L} = - \sum_t \log P(y_t \mid y_{<t}, X)$$

CROSS-ENTROPY LOSS

Summed over all target token positions t ; minimized via Adam optimizer

Key architectural numbers: $d_{\text{model}} = 512$, $h = 8$ heads, $d_k = d_v = 64$, $d_{\text{ff}} = 2048$, $N = 6$ layers, dropout = 0.1, label smoothing $\epsilon = 0.1$. Total parameters: ~65M (base model), ~213M (large model).

Adam optimizer

Warmup LR schedule

Label smoothing

Dropout = 0.1